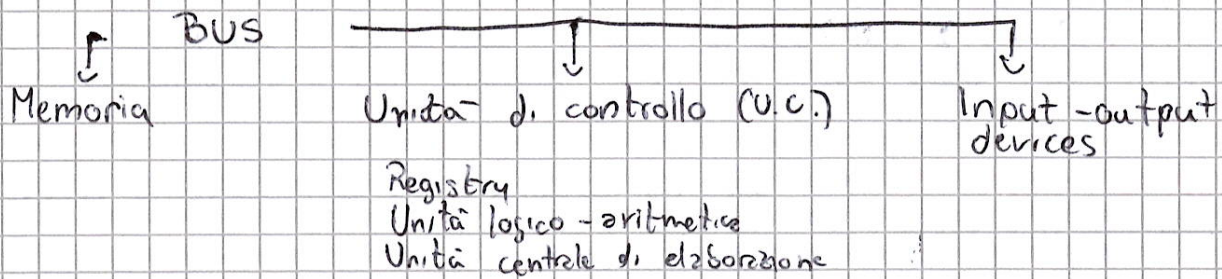


MODELLO DI VON-NEUMANN



MODELLO è una rappresentazione concettuale (spesso una semplificazione) del mondo reale o di una sua parte.
○ di una semplificazione atto a spiegare il funzionamento.

Il modello di Von-Neumann comprende:

- Un **UNITÀ DI CONTROLLO** che sorveglia le operazioni del calcolatore, invia segnali agli altri componenti e interpreta le istruzioni della memoria.
- Un **UNITÀ LOGICO-ARITMETICA** che implementa il core logici e permette di prendere decisioni, assieme al permettere di effettuare operazioni aritmetiche di base.
- La **MEMORIA** che conserva le istruzioni e i dati da inviare alla CPU e restituiti dall'elaborazione di esse.
- **UNITÀ DI INPUT** che permette di immettere informazioni nel calcolatore, chiamate anche **PERIFERICHE**.
- **UNITÀ DI OUTPUT** con cui le informazioni e le elaborazioni del calcolatore vengono fatte "uscire" dal calcolatore.

Spesso, l'ALU e l'UC sono integrate nell'unità centrale di elaborazione **CPU**.

La memoria è composta da indirizzi di locazione, contenuto delle celle e le celle. Solitamente la CPU chiede un dato fornendone il suo indirizzo, e la memoria risponde con il dato.

La CPU contiene al suo interno dei **REGISTRI**, che possono essere visti come:

- Registro ISTRUZIONE, con la sua esecuzione (R.I.)
- Registro contatore, con l'address in memoria della prossima istruzione (P.C.)
- Registro DATI

Esecuzione

Il program counter comunica al M.A.R. l'indirizzo dell'istruzione, poi si incrementa di uno.

Aspettare che la memoria reperisce il contenuto e lo pone disponibile sul M.D.R.
Copiare il contenuto del M.D.R. nell'(R.I.)

LINGUAGGIO MACCHINA

Il linguaggio macchina è ciò che la CPU sa eseguire, ed è dipendente da ogni calcolatore. Esso, scritto in forma BINARIA, non è comprensibile dall'uomo.

IL LINGUAGGIO ASSEMBLY è una corrispondenza 1:1 con il linguaggio macchina, ed è leggibile dall'uomo.

2. I linguaggi spesso sono composti da istruzioni non elementari, che spesso, ogni istruzione corrisponde a PIÙ istruzione in linguaggio macchina.

Solitamente si scrive in linguaggio di PIÙ ALTO LIVELLO (es. Python, Java, C) del linguaggio assembly.

La VELOCITÀ si misura in operazioni semplici al secondo, [Hertz]

LE LEGGI DI MOORE

1. Il numero di transistor per mm aumenta raddoppiandosi ogni 24 MESI per aumentare il numero di transistor per mm bisogna rimpicciolirli.

da 1 μ m (micrometri) del 2011 si è passati a 7 nm su amd per la prima volta

c. Si sta avvicinando a limiti quantistici.

- la luce, in un nanosecondo percorre 30 cm.

- se il chip è di 36 cm allora non

- e la frequenza di clock è di 3 GHz vuol dire che la luce in un ciclo percorre 10 cm.

- il calore sviluppato è più alto

MULTI CORE! calcolo parallelo su più chip.

RAPPRESENTAZIONE DI CARATTERI, STRINGHE E ALTRI DATI

Il calcolatore manipola solamente "0" e "1", quindi tutti i dati devono essere codificati in maniera tale che ad una sequenza di 0 e 1 viene corrisposto un valore

Ogni carattere è rappresentato.

ad una sequenza univoca di 8 bit.

ASCII consiste in una codifica di 2^7 combinazioni di dati.

es "Z" = 1111010 (numero 122)

la codifica consiste in 128 combinazioni (sequenze) da 7 bit.

N.B. è possibile usare 8 codificando e ponendo uno 0 all'ottavo numero a partire da destra.

Ci sono alcuni numeri che non rappresentano caratteri.

es

1 NUL fine stringa

9 TAB tabulazione

10 LF Line feed nuova riga.

127 DEL

LA CODIFICA ASCII è LIMITATA, e mancano molti simboli particolari "è".
esistono quindi dei nuovi standard

- ISO 8859 (de International Standard Organization).
esso è un insieme di STANDARD (di 16 versioni)

è indicata con ISO 8859-i (con i da 1 a 16).
Ogni versione codifica 2^8 caratteri. (128)

1. la versione 1 è Latin Western European, più usata
non basta perché se si vogliono usare due versioni non è possibile.

LO UNICODE (o UCS)

comprende tre possibili codifiche, che semplicemente utilizzano più o meno bit per carattere

- UTF-8, utilizza fino a 4 unità da 8 bit (più usata) - per più efficiente
- UTF-16 utilizza fino a 2 unità da 16 bit
- UTF-32 utilizza sempre ogni carattere

UTF 8.

- i caratteri che usano un solo byte (1 unità da 8 bit) sono quelli ASCII
e si riconoscono dalla presenza di uno zero in binario
(quello in più del 2^7)

NUM	byte 1	1	2	3	4
da 0	0xxx xxxx				
128	110x xxxx	10xx xxxx			
2	1110 xxxx	10xx xxxx	10xx xxxx		
2048	1111 xxxx	10xx xxxx	10xx xxxx	10xx xxxx	
65535	1111 0xxx	10xx xxxx	10xx xxxx	10xx xxxx	10xx xxxx

in una stringa ogni
carattere equivale
ad un numero

Si usa lo 0 per il fine riga

RAPPRESENTAZIONE DI NUMERI INTERI -senza segno-

come nel sistema decimale, si possono rappresentare numeri con il sistema binario, ottale ed esadecimale

dec.	0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15
ott	0	1	2	3	4	5	6	7	10	11	12	13	14	15	16	17
hex	0	1	2	3	4	5	6	7	8	9	A	B	C	D	E	F

il sistema posizionale le singole cifre rappresentano se stesse,

ad esempio

il numero 20 in ottale
è 2 volte la cifra in posizione 1,
che vale 8. quindi $2 \cdot 8 = 16$ decimale

nel caso del 105 in base 8 vuol dire

$$1 \cdot 8^2 + 2 \cdot 8 + 5 = 64 + 16 + 5 = 85$$

quindi, in base x è

$$ax^n + bx^{n-1} + cx^{n-2} + \dots + ex^0$$

Se la base b_1 è potenza esatta della base b_2
allora ogni cifra della base b_1 sono in gruppo,

NUMERI INTERI CON SEGNO

2 modi

- bit di segno
- complemento alla base

BIT di segno è il primo bit (è partine de se per decide il segno)

1 = negativo

0 = positivo

gli altri bit sono di valore assoluto

$$1011 = 011 \text{ modulo } \text{negativo} = -3$$

lo svantaggio è che per fare operazioni bisogna distinguere i casi
es

Somme $1011 + 0101$

avendo segni opposti bisogna fare operazione di differenza.

$$\begin{array}{r} 101- \\ 011- \\ \hline 000 \end{array} \Rightarrow 0010$$

per risolvere a questo problema si usano i **COMPLEMENTI**

• I numeri positivi si rappresentano nello stesso modo, e con 0 di segno.

• I numeri **NEGATIVI** hanno bit di segno **1** e ogni bit è invertito e si somma 1

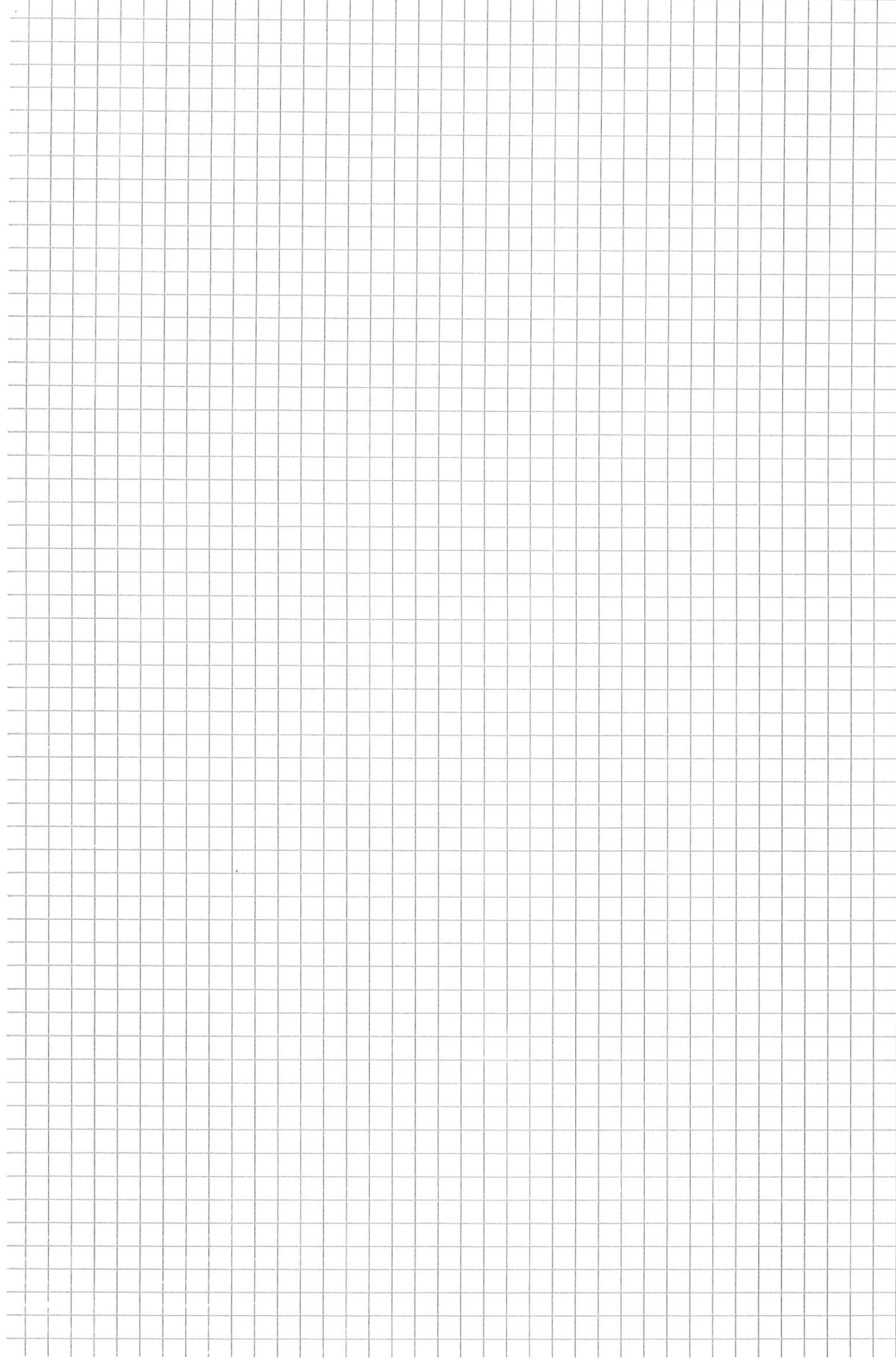
es. $0010 = \text{complemento } 1101 + 1 = 1110$

altro metodo è partendo da destra si cerca il primo 1, da lì si inverte tutto.

0110

↑ primo 1, si inverte da dopo di questo 1010

uno	001	→	- uno	111
due	010	→	- due	110



)

)

)

)



RAPPRESENTAZIONE INTERI SU PYTHON

La rappresentazione di un numero è composta da una variabile che denota numero di blocchi e della serie di bit da 30

Ob-size 3

Ob-digit $(a)_2 (b)_2 (c)_2$

per riconvertire il numero si fa

$$(a \cdot 2^{30 \cdot 0}) + (b \cdot 2^{30 \cdot 1}) + (c \cdot 2^{30 \cdot 2})$$

NUMERI FRAZIONARI

$0, C_1 C_2 C_3 C_n \dots$

$$C_1 \cdot 10^{-1} + C_2 \cdot 10^{-2} + C_3 \cdot 10^{-3} + C_n \cdot 10^{-n} \dots$$

Per trasformare da base n a decimale si fanno i calcoli:

$$0,685 = 6 \cdot 10^{-1} + 8 \cdot 10^{-2} + 5 \cdot 10^{-3} = \frac{6}{10} + \frac{8}{100} + \frac{5}{1000} = 0,685$$

Da DECIMALE a ALTRA BASE, es

0,6875 in base 2

$$0,6875 \times 2 = 1,375$$

$$0,375 \times 2 = 0,75$$

$$0,75 \times 2 = 1,5$$

$$0,5 \times 2 = 1 \text{ si finisce}$$

} 0,1001

Questa procedura non sempre termina. A volte succede una periodicità, che non può essere rappresentabile

$$\text{es)} \quad 0,2 \times 2 = 0,4$$

$$0,4 \times 2 = 0,8$$

$$0,8 \times 2 = 1,6$$

$$0,6 \times 2 = 1,2$$

$$0,2 \times 2 = 0,4$$

$$\dots$$

0,0110011...

Ci saranno altre approssimazioni.

esempio 0,1 non è esatto, ma approssimato

$$0,1 \times 2 = 0,2$$

$$0,2 \times 2 = 0,4$$

$$0,4 \times 2 = 0,8$$

$$0,8 \times 2 = 1,6$$

$$0,6 \times 2 = 1,2$$

$$0,2 \times 2 = 0,4$$

0,00011001100110011001100

da decimale a altra base si moltiplica $C_i \cdot 2$ e si prende la parte intera come cifra, la virgola come es. (MOLTIPLICAZIONI) successive

Virgola mobile IEEE 754

viene usato il simbolo # per indicare la posizione della virgola, usando un metodo per indicare numeri molto grandi o piccoli:

$$354\#0 = 0,354 = 0,354 \cdot 10^0$$

$$75345\#2 = 75,345 = 0,75345 \cdot 10^2$$

il numero prima dell'Hash '#' è la mantissa.

Calcoli con Mantisse e virgole mobili:

$$101\#3 + 2\#2$$

Si modifica il numero con esponente più piccolo in modo che abbia lo stesso esponente dell'altro

Si sommano le mantisse

Se il risultato è maggiore o uguale a uno si divide per 10 e si aumenta l'esponente di 1.

$$2\#2 = 0,2 \cdot 100 = 20 \text{ ma anche } 0,02 \cdot 1000$$

$$\text{quindi, } 0,101 \times 1000 + 0,02 \times 1000 = 0,121 \cdot 1000 = 121\#3$$

Un formato per rappresentare i numeri in virgola mobile è IEEE 754

1 bit di segno

11 bit di esponente

52 bit di mantissa

Il numero secondo IEEE 754 è di forma

$$1, C_1 C_2 C_3 \times 2^{\text{esponente}}$$

$$1100110 \rightarrow 1,100110$$

$$0,00010111 \rightarrow 1,0111$$

per esempio 349,625

Si converte in binario parte intera con moltiplicazioni/divisioni successive e parte frazionaria con moltiplicazioni successive

$$101011101,101$$

trovare la forma 1, cifra, quindi $1,01011101101 \dots 2^8$

la mantissa è 01011101101

per trovare l'esponente si memorizza $1023+8 = 1031$ in binario:

$$10000000111$$

Si sommano per differenziare positivi e negativi esponenti

quindi è

$$\underbrace{0}_{\text{segno}} \underbrace{10000000111}_{\text{Esponente}} \underbrace{010111011101}_{\text{mantissa}} + 0,61 \text{ volte}$$

tipo double
64 bit

RAPPRESENTAZIONE DI NUMERI INTERI (senza segno)

I numeri sono rappresentati posizionali in quanto qualsiasi numero

es $1257_{(10)}$ è esprimibile come $7 \cdot 10^0 + 5 \cdot 10^1 + 2 \cdot 10^2 + 1 \cdot 10^3$
 $7 + 50 + 200 + 1000$

Gli esponenti vanno ad indicare la POSIZIONE, le basi indicano la BASE
 del SISTEMA NUMERICO

es $101011_{(2)} = 1 \cdot 2^0 + 1 \cdot 2^1 + 0 \cdot 2^2 + 1 \cdot 2^3 + 0 \cdot 2^4 + 1 \cdot 2^5$

Si considera quindi che con "C" esponente e "b" la sua base

$$C_k C_{k-1} \dots C_1 C_0 = C_0 \times b^0 + C_1 \times b^1 + \dots + C_{k-1} \times b^{k-1} + C_k \times b^k$$

CONVERSIONE IN DECIMALE

basta sviluppare la regola applicandola in base 10

es $42703_{(8)} = 4 \cdot 8^4 + 2 \cdot 8^3 + 7 \cdot 8^2 + 0 \cdot 8^1 + 3 \cdot 8^0 =$
 $= 16384 + 1024 + 448 + 0 + 3 =$
 $= 17853_{(10)}$

CONVERSIONE DA DECIMALE

- si calcola quoziente e resto in divisore b
- il resto è l'ultima cifra in base b (più a destra)
- quoziente si divide per b
- resto è penultima cifra.

es
 D2 $319_{(10)} \rightarrow$ base 2

$319/2 = 159$	resto 1	Cifre C_0
$159/2 = 79$	" 1	" C_1
$79/2 = 39$	" 1	" C_2
$39/2 = 19$	" 1	" C_3
$19/2 = 9$	" 1	" C_4
$9/2 = 4$	" 1	" C_5
$4/2 = 2$	" 0	" C_6
$2/2 = 1$	" 0	" C_7
$1/2 = 0$	" 1	" C_8

$$(319)_{(10)} = (100111111)_{(2)} \quad (2) \quad (2)$$

CONVERSIONE DA BASE A MULTIPLI

es.
 3 cifre in base 2 sono 1 cifra in base 8 $(8 = 2 \times 2 \times 2)$

es. $C_5 C_4 C_3 C_2 C_1 C_0 = C_5 \times 2^3 + C_4 \times 2^2 + C_3 \times 2^1 + C_2 \times 2^0 + C_1 \times 2^4 + C_0 \times 2^3$
 $= (C_5 \times 2^2 + C_4 \times 2^1 + C_3) \times 2^3 + \underbrace{C_2 \times 2^2 + C_1 \times 2^1 + C_0}_{D_0} \times 2^0$
 prime cifre ottali è d_1
 seconde è d_0

es $(11010111001)_2 = \begin{matrix} 011 & 010 & 111 & 001 \\ 3 & 2 & 7 & 1 \end{matrix} = (3271)_8$

DA BASE A DIVISORI

es Ogni cifra ottale sono 3 bit binari

$(52172)_8 = \begin{matrix} 5 & 2 & 1 & 7 & 2 \\ 101 & 010 & 001 & 111 & 010 \end{matrix} = (101010001111010)_2$

ADDIZIONI E SOTTRAZIONI

funzionano come la base decimale

es Somme base 8

$$\begin{array}{r} 1 \ 1 \ 1 \\ 523+ \\ 1361 \\ \hline 2104 \end{array}$$

riporto 0
riporto 1
riporto 1
riporto 0

$= (2104)_8$

MOLTIPLICAZIONE

funzionano grazie a metodo posizionale

$a \times C_k \dots C_0 = a \times C_k \times b^k + \dots + a \times C_0 \times b^0$

il numero $a \times C_i$ rappresenta $d_i = d_n \dots d_0 \times b_n \dots \times b_0$

va moltiplicato per ogni b^i che quindi è diventato $d_n \times b^{n+i} + \dots + d_0 \times b^i$

$$\begin{array}{r} 10111 \times \\ 1101 = \\ \hline 10111 \\ 00000 \\ 10111 \\ 10111 \\ \hline 100101001 \end{array}$$

10111×0
 10111×0
 $\dots \times 1$
 $\dots \times 1$

Ogni prodotto è ~~300~~ oppure il primo operando spostato a sinistra

OVERFLOW (o underflow)

Ogni numero può essere rappresentato con un finito numero di bit.
Se un operazione porta ad un numero di bit più alto dello spazio disponibili, il numero è perso

CALCOLO PROPOSIZIONALE

Si è studiato la rappresentazione di strutture di dati in binario, ora è importante vedere come si possono realizzare queste cose in circuiti elettronici

$$\begin{array}{r} \text{es} \quad \begin{array}{r} 1011+ \\ 1101= \\ \hline 11000 \end{array} \end{array}$$

Somma per singola cifra, ogni possibile cifra

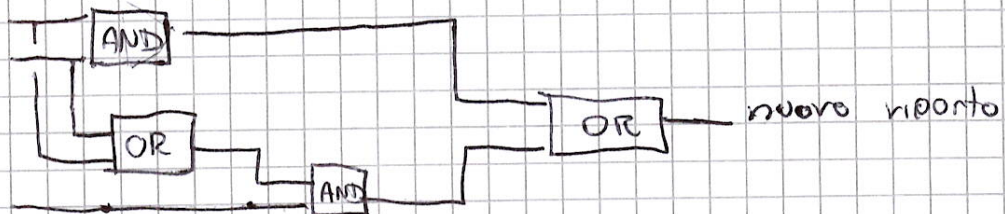
riporto	prime	seconda	risultato	riporto
0	0	0	$\Rightarrow 0$	0
0	0	1	$\Rightarrow 1$	0
0	1	0	$\Rightarrow 1$	0
0	1	1	$\Rightarrow 0$	1
1	0	0	$\Rightarrow 1$	0
1	0	1	$\Rightarrow 0$	1
1	1	0	$\Rightarrow 0$	1
1	1	1	$\Rightarrow 1$	1

il riporto vale 1 se:

- le 2 cifre verso sinistra 1
- il riporto vale 1 e una delle due cifre vale 1

È UNA FUNZIONE BOOLEANA

funzione trasformata in circuito elettrico



Esistono ~~tre~~ componenti che realizzano una o più porte logiche

esempio è componente "holl" con ~~una~~ porte NAND (not-and)

(25)

a	b	c	?
0	0	0	0
0	0	1	0
0	1	0	1
0	1	1	1
1	0	0	1
1	0	1	0
1	1	0	0
1	1	1	0

$$a=0 \Rightarrow \text{NOT } a$$

$$a=1 = a$$

$$b=0 \Rightarrow \text{NOT } b$$

$$b=1 = b$$

$$c=0 \Rightarrow \text{not } c$$

$$\begin{aligned} & ((\text{NOT } a) \text{ AND } b \text{ AND } (\text{NOT } c)) = \text{linea } 010 \\ & ((\text{NOT } a) \text{ AND } b \text{ AND } c) = \text{linea } 011 \\ & (a \text{ AND } (\text{NOT } b) \text{ AND } (\text{NOT } c)) = \text{linea } 100 \\ & (a \text{ AND } (\text{NOT } b) \text{ AND } c) = \text{linea } 101 \end{aligned}$$

Si guardano i primi due casi (ultimi)

$$(a \text{ AND } (\text{NOT } b) \text{ AND } (\text{NOT } c)) \text{ OR } (a \text{ AND } (\text{NOT } b) \text{ AND } c)$$

ci sono 2 possibilità

$$1. \Rightarrow \text{AND } (\text{NOT } b) \text{ AND } (\text{NOT } c)$$

$$2. \Rightarrow \text{AND } (\text{NOT } b) \text{ AND } c$$

Sono uguali le prime due istruzioni, e non importa lo stato di c

$$\text{quindi} \Rightarrow \text{AND } (\text{NOT } b)$$

Stesse cose per le prime possibilità vere

$$\text{NOT } a \Rightarrow \text{AND } b \text{ AND } \text{NOT } c \text{ or } \text{NOT } a \Rightarrow \text{AND } b \text{ AND } c$$

$$\text{quindi, } \text{NOT } a \text{ AND } b$$

⇓

$$((\text{NOT } a) \text{ AND } b) \text{ OR } (a \text{ AND } (\text{NOT } b)) \text{ Semplificazione}$$

Servono algoritmi per semplificare formule, con la LOGICA PROPOSIZIONALE

valori = falso, vero (0,1)

variabili

connettivi: $\neg \wedge \vee$ (NOT, AND, OR)

regole:

- COMMUTATIVA

$$a \wedge b = b \wedge a \quad \text{oppure} \quad a \vee b = b \vee a$$

- ASSOCIATIVA

$$(a \wedge b) \wedge c = a \wedge (b \wedge c) \quad \text{oppure} \quad (a \vee b) \vee c = a \vee (b \vee c)$$

- DISTRIBUTIVA

$$a \wedge (b \vee c) = (a \wedge b) \vee (a \wedge c)$$

$$a \vee (b \wedge c) = (a \vee b) \wedge (a \vee c)$$

Un'interpretazione è una funzione assegnando verità o falsità a ciascuna variabile.

$$(a \vee \neg b) \wedge c \wedge \neg a \quad a=1 \quad b=0 \quad c=0$$

$$1 \wedge 0 \wedge 0 = 0$$

resta 1 solo 0 su AND per fare tutti 0

Un'interpretazione che rende VERA una formula è il MODELLO

I modelli servono a definire:

- a2

METODI DI RISOLUZIONE

$$(A \vee \neg B) \wedge \neg A \wedge (B \vee A)$$

trovare i valori per cui questa funzione è vera

$$(F \vee C) \wedge (F \vee \neg C) = F \quad \} \text{Formula!}$$

$$(A \vee \neg B) \wedge \neg A \wedge (B \vee A) =$$

$$(A \vee B) \wedge \neg A \wedge (B \vee A) \wedge A = 0$$

$$\downarrow \rightarrow \neg A \wedge A \downarrow$$

NON POSSIBILE, contraddizione

La risoluzione utilizza le proprietà

$$(F \vee \neg b) \wedge (D \vee b) = F \vee D \quad \text{aggiunta}$$

per usare la forma la formula dev'essere impostata con concatenazione AND con termini con solo or

"A implica B"

$A \models B$ & ogni interpretazione per cui $A=1$ è tale per $B=1$

NON ESISTE un'interpretazione per cui $A \neq B$

non soddisfabilità di $A \wedge \neg B$

DUE FORMULE SONO EQUIVALENTI se l'implicazione è anche inversa

Def: **LA SODDISFACIBILITÀ** è il metodo con cui si vede se una determinata funzione di logica può o non può avere un modello

Regola generale di risoluzione del tipo

$$(a \vee c \vee \neg b) \wedge (b \vee d) \wedge \dots$$

Si applica la regola di risoluzione:

$$(Funzione F \vee \neg b) \wedge (b \vee Funzione D) \Rightarrow \text{si aggiunge } \text{FAD} \text{ FUD}$$

IMPLICAZIONI

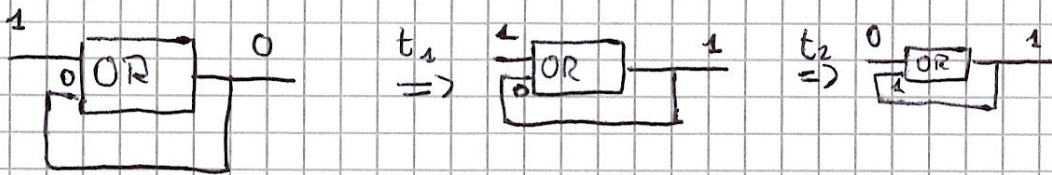
Equivalenze di De Morgan Utili

$$\neg(a \vee b) = \neg a \wedge \neg b$$

$$\neg(a \wedge b) = \neg a \vee \neg b$$

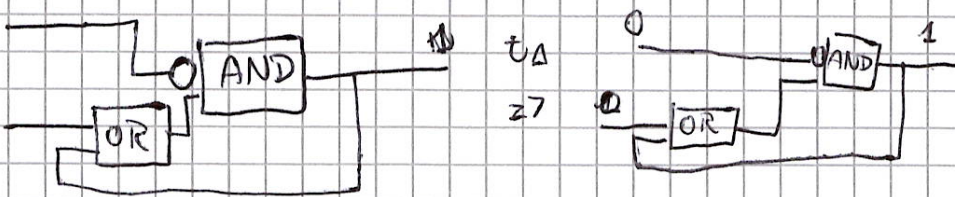
RETROAZIONE

- Si usa il tempo per MEMORIZZARE St di memoria attraverso
 2 retroazione

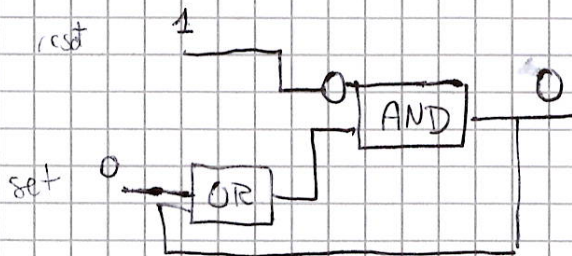


che l'input sia 0 o 1
 l'output sarà sempre 1.

Si può RESETTARE attraverso un flip-flop



Si resetta applicando voltaggio sull'altro pin



Output =

$$= \text{not RESET} \wedge (\text{set} \vee \text{output})$$

CALCOLABILITÀ

Alcuni problemi più complicati hanno bisogno di un algoritmo risolutivo e quindi programmi che lo risolvono.

ma ogni problema può essere risolto?

Considerazioni:

I modelli di calcolo: Sono insiemi di regole non ambigue ed eseguibili utilizzati per definire un algoritmo.

PYTHON È UN ^{MODELLO} METODO DI CALCOLO

Un modello può essere più o meno espressivo: i.e. più o meno potente/funzionale
esistono soluzioni a 0 o 1, (vero o falso)

«Sono» nei problemi decisionali (vero o falso) anche
- L'appartenenza di un numero IN ad un insieme

Alcuni problemi non hanno calcolabilità e quindi non sono risolvibili.

es.

- File python che LEGGE se stesso
- Problema della fermata: - programma NON TERMINA se $\exists$ l'input non $\bar{c}$ che non lo fa terminare

Es

```
while i != 10:  
    print(i)  
    i += 1
```

Se l'input è maggiore di 10 non termina mai

Non esiste un programma che verifica se un programma che termina o no

Per dimostrare ciò si usa l'autoreferenzialità: Se esiste un programma esiste anche quello inverso.

Collegano al concetto del PARADOSSO

COMPLESSITÀ

L'efficienza:

l'uso del minor numero di risorse possibili per un dato compito.
Con RISORSE si intende -in questo corso- il tempo e lo spazio.

Si misura solo la stima GENERALE, non il tempo esatto

Si usa la notazione $O(n)$ per indicare la componente che impatta più nel programma

Un programma ha un costo $O(f(n))$ se il suo tempo di esecuzione EFFETTIVO $t(n)$ ed esistono opportune costanti a, b , e n_0 tali che

$$t(n) < a \cdot O(f(n)) + b$$

per ogni $n > n_0$

con n la grandezza di input

Il tempo di esecuzione s , misura come NUMERO DI ISTRUZIONI SEMPLICI che vengono eseguite.

Con la notazione $O(f(n))$ si considera il possibile tempo più grande, con il caso peggiore

Trovare il costo in un programma è anche trovare le opportune a, b e n_0

es $a = [3, 9, -1, 4, 3]$

$S = 0$

for x in a :

$S = S + x$

print(S)

non usiamo il metodo sum

perché il costo è calcolato

su ~~alle~~ istruzioni ELEMENTARI.

Esiste l'istruzione DOMINANTE, che sono una qualsiasi delle istruzioni che vengono eseguite più volte.

Nel caso dell'esempio l'inizializzazione della variabile a della lista ha costo 1.

il ciclo for viene eseguito 1 volta sola

La somma viene eseguita 5 volte! trovate l'istruzione dominante.

Se la lista avesse n numeri l'istruzione dominante viene eseguita n volte

Il costo dell'istruzione del programma è il costo dell'istruzione DOMINANTE

Nel caso di presenza di cicli il costo dell'istruzione dominante è quello del ciclo più interno.

ESPRESSIONI REGOLARI

Sono modelli matematici che includono:

- definizione di stringa [concatenazione di simboli di alfabeto]
- sintassi di espressioni regolari
- Semantica

Il linguaggio è un insieme di STRINGHE:

- non sono elencabili tutte! (spesso infinite!)

- bisogna quindi trovare un modello di linguaggio che collima o non collima

STRINGHE:

collima = soddisfa l'espressione

Non collima = non soddisfa l'espressione

- Sono costruite con

• Concatenazione: date s e r la loro concatenazione (sr) è un'altra stringa contenente la successione della stringa s e la r

• UNIONE: $A \cup B$

• INTERSEZIONE $A \cap B$

• INSIEME VUOTO, diverso dalla STRINGA VUOTA (ϵ)

Semantica

ϵ = stringa vuota

e^* = Indica 0 o più occorrenze di e

$e|f$ = e oppure f

$e?$ = e opzionale (0 oppure 1 occorrenze)

La semantica è una funzione matematica collimano(ϵ), [match]

es

• collimano(abc) = $\{ab, abc\}$

• collimano(a^*b) = $\{b, ab, aab, aaab, \dots\}$

• collimano($(ab^?)(c^*)$) = $\{a, ab, \epsilon, c, cc, ccc, \dots\}$

Abbreviazioni

$[X_1 \dots X_n]$ è abbreviazione di $X_1|X_2|\dots|X_n$

$[X-Y]$ caratteri compresi tra X e Y (infatti l'alfabeto è ordinato)

$^1[X-Y]$ tutti i caratteri TRUOVE quelli compresi tra X e Y

e^+ occorrenza del carattere in 1 o più volte (e^*)

$e\{n\}$ collima con la stringa composta da n volte il carattere

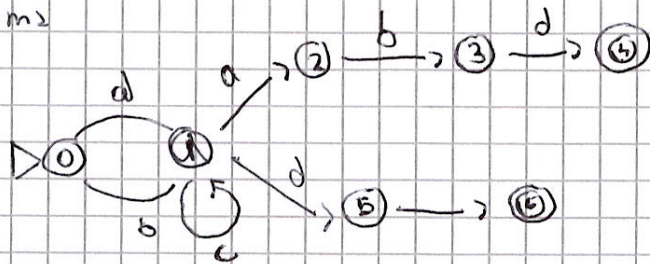
$e\{n, m\}$ collima con le ripetizioni di e da n (minimo) a m (massimo)

AUTOMI

Corrispondenza reges

$[ab]c^*(abc|dd)$

Automa



carattere fra a e b, numero qualsiasi (anche 0) di c,
poi abc oppure dd

Se dallo stato 0 di inizio trovo a oppure b
passo allo stato 1

Se da 1 trovo c torno nello stesso stato 1

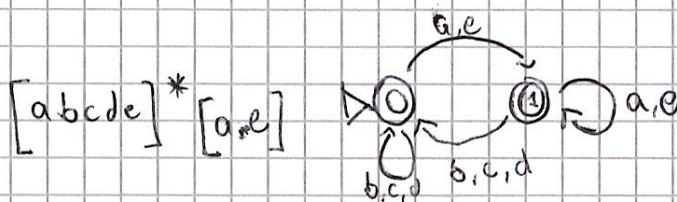
" " " trovo a si parte nello stato 2

" " " d si parte sullo stato 5

La computazione termina o al termine dell'automa oppure
quando viene introdotto un carattere Non DEFINITO (in cui non c'è uscita
di stato)

Definizioni:

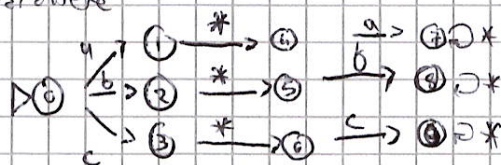
- Stati (che contengono il numero stato)
- Archi (collegano gli stati)
- Loop = archi che portano allo stesso stato
- Stato finale (cerchio doppio)



tutte le parole composte
da abcde che terminano
con a,e

Stato - memoria = stato diverso e secondo di altre condizioni o di altri caratteri

si usa l'asterisco per indicare qualsiasi altro
carattere



AUTOMI E PORTE LOGICHE

Formalmente, un automa è una QUINTUPLA $\langle I, S, S_0, A, N \rangle$

I insieme degli ingressi possibili (lettere dell'alfabeto)

S insieme degli stati

S_0 elemento iniziale di stato

A un sottoinsieme di S , degli stati finali

N una funzione $S \times I \rightarrow S$ (prodotto cartesiano), funzione di stato successivo

